Afnamtech Private Limited

Website Project Documentation

Project: Afnamtech Private Limited Company Website

Architecture: Static frontend + Node.js local server + Netlify Serverless Functions

Email: Nodemailer + SMTP

Database: No database integration is present in the supplied project

This documentation is prepared from the supplied Afnamtech Private Limited website project and follows the structure and detail level of the provided ABC Solutions documentation example.

1. Project Overview

The Afnamtech Private Limited Website is a corporate web application designed to present the company's banking technology services, solutions, products, delivery capabilities, career opportunities, and contact channels. Visitors can browse company information, explore banking-focused solutions, view available job openings, submit contact enquiries, and apply for positions with an optional resume attachment.

The supplied project is primarily a static website built with HTML, CSS, and JavaScript. Server-side functionality is implemented through Netlify Functions, while a lightweight Node.js HTTP server is included for local development. Email delivery is handled with Nodemailer over SMTP.

Technology Stack

Layer	Technology / Component
Frontend	HTML5, CSS3, Vanilla JavaScript
Additional UI	React 19 components and an esbuild bundle
Local backend	Node.js built-in HTTP server
Serverless backend	Netlify Functions
Email delivery	Nodemailer with SMTP
Build tool	esbuild
Environment configuration	dotenv / .env
Testing	Node.js built-in test runner
Deployment configuration	Netlify

Project Characteristics

- Five primary public pages are included: Home, Our Story, Solutions, What We Offer, and Career.
- The contact form sends a notification to the configured company mailbox and a confirmation to the visitor.
- The career page retrieves job listings from a serverless API and submits applications through a serverless endpoint.
- Career applications can include PDF, DOC, or DOCX resumes up to 5 MB.
- No MongoDB, MySQL, PostgreSQL, or other database connection is present in the supplied project.
- Job listings and solution data are currently defined in JavaScript arrays rather than stored in a database.

2. Website Pages

Home Page

File: public/index.html

- Introduces Afnamtech Private Limited and its banking technology positioning.
- Hero content includes 'Innovating Your Future', Digital Platform, and Artificial Intelligence messaging.
- Presents products and banking capabilities including CRM, workforce/HRMS, digital banking, core banking, and data migration.
- Contains case-study and delivery-story content.
- Includes vision, mission, engagement models, leadership, partners, clients, and technology/tooling sections.
- Provides the 'Get in touch' contact interaction and contact form.

Our Story Page

File: public/our-story.html

- Presents the company story and positioning as an IT services partner for banks and financial institutions.
- Contains a company timeline covering milestones from 2019 through 2025.
- Uses JavaScript/IntersectionObserver behavior for milestone-related visual interaction.

Solutions Page

File: public/solutions.html

- Presents six banking/financial-services solutions.
- Solutions are AML, Ticketing System, ChatBot, HRMS (WS Workforce), Payment Automation, and CRM (WS CRM).
- The page uses JavaScript to expand and collapse solution content.

What We Offer Page

File: public/what-we-offer.html

- Presents three service pillars: Digital Banking Solutions, Core Banking Solutions, and Data Migration.
- Includes technology/tooling information and engagement models such as Fixed Price and Time & Material.

Career Page

File: public/career.html

- Loads available jobs dynamically from GET /api/careers.
- Provides category and work-mode filtering in the browser.
- Provides an application form with name, email, phone, role, department, cover letter, and resume fields.
- Submits applications to POST /api/careers-apply.

3. Solutions and Career Information

Solutions API Data

No.	Solution	Main Capabilities
1	AML (Anti-Money Laundering)	KYC/risk profiling, transaction monitoring, SAR, watchlist screening, case management, regulatory reporting
2	Ticketing System	Audit/query workflows, SLA tracking, reports, timestamped tracking
3	ChatBot	Mobile/web integration, 24x7 service, multi-channel and multi-language support, AI/ML, cloud/on-premise
4	HRMS (WS Workforce)	Employee management, attendance/leave, payroll, recruitment, compliance, mobile access, analytics
5	Payment Automation	Incoming credits, acknowledgements, returns, API integration, reconciliation, reporting
6	CRM (WS CRM)	Customer information, communication history, sales pipeline, mobile access, role-based access, scalability

Available Career Positions

Role	Department	Type	Location
Core Banking Integration Engineer	Engineering	Full-time	Dharmapuri (On-site)
React Frontend Developer	Engineering	Full-time	Remote
Business Analyst - Banking Domain	Banking	Full-time	Dharmapuri (On-site)
HR Executive	HR	Full-time	Dharmapuri (On-site)

Career API Response Structure

```
{
  "success": true,
  "data": [
    {
      "role": "React Frontend Developer",
      "department": "Engineering",
      "type": "Full-time",
      "location": "Remote",
      "isRemote": true,
      "description": "...",
      "requirements": ["React", "REST API integration", "CSS"]
    }
  ]
}
```

The job data is defined directly inside netlify/functions/careers.js. There is no database-backed job management system in the supplied project.

4. Forms and Email Workflow

Contact Form

- Location: Home Page.
- Purpose: Collect customer enquiries and messages.
- Frontend endpoint: POST /api/contact.
- Required fields: Full Name, Email, Subject, Message.
- The server validates the request before sending any email.
- A successful submission sends two emails: a company notification and a visitor confirmation.

Contact Email Flow

```
Visitor
   |
   v
Contact Form
   |
   | POST /api/contact
   v
Netlify Function: contact.js
   |
   +----> COMPANY_EMAIL
   |       New Contact Enquiry
   |
```

```
        +----> Visitor Email
               Confirmation Message
```

Career Application Form

- Location: Career Page.
- Purpose: Collect candidate applications.
- Frontend endpoint: POST /api/careers-apply.
- Required fields: Full Name, Email, Phone, Role.
- Optional fields: Department, Cover Letter, Resume.
- Resume types accepted by server-side validation: PDF, DOC, DOCX.
- Maximum resume size: 5 MB.
- The resume is sent as an email attachment to the configured company mailbox.
- A confirmation email is also sent to the applicant.

Career Application Email Flow

```
Candidate
   |
   v
Career Application Form
   |
   | POST /api/careers-apply
   v
Netlify Function: careers-apply.js
   |
   +----> COMPANY_EMAIL
   |       Job Application + Resume
   |
   +----> Candidate Email
           Application Received Confirmation
```

5. SMTP Email Configuration

The supplied project uses Nodemailer with SMTP. It does not use EmailJS in the current implementation.

Environment Variables

Variable	Purpose	Example
SMTP_HOST	SMTP server address	smtp.gmail.com
SMTP_PORT	SMTP port	587
SMTP_USER	Account used to send email	your_gmail@gmail.com
SMTP_PASS	SMTP password or provider App Password	16-character Gmail App Password
COMPANY_EMAIL	Mailbox receiving company notifications/applications	info@yourcompany.com

Example .env File

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your_gmail@gmail.com
SMTP_PASS=your_16_character_app_password
COMPANY_EMAIL=info@yourcompany.com
```

Gmail App Password Setup

- Enable 2-Step Verification on the Google account used for SMTP.

- Create a Google App Password for the website/application.
- Use the generated App Password as SMTP_PASS; for Gmail, do not use the normal account password for this configuration.
- Keep the .env file private and never commit it to Git.

SMTP Transport

```
const transporter = nodemailer.createTransport({
  host: SMTP_HOST,
  port: SMTP_PORT,
  secure: SMTP_PORT === 465,
  auth: {
    user: SMTP_USER,
    pass: SMTP_PASS
  }
});
```

6. Running the Project Locally

Step 1: Install Dependencies

```
npm install
```

Step 2: Create the Environment File

```
copy .env.example .env

# macOS/Linux:
cp .env.example .env
```

Step 3: Configure SMTP

Open .env and provide SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, and COMPANY_EMAIL.

Step 4: Build the React Widget Bundle

```
npm run build
```

Step 5: Start the Local Server

```
npm start
```

The local server uses port 3000 by default. Open:

```
http://localhost:3000
```

Available npm Scripts

Command	Purpose
npm install	Install project dependencies
npm start	Start the local Node.js HTTP server
npm run dev	Start the local server in development mode
npm run build	Bundle the React widget source with esbuild
npm run netlify-build	Run the Netlify build command
npm run lint	Run Node syntax checks on key backend files
npm test	Run the Node.js test suite

Local Routes

Route	Page

/	Home page
/our-story	Our Story page
/solutions	Solutions page
/what-we-offer	What We Offer page
/career	Career page

7. API Endpoints

Endpoint	Method	Purpose	Implementation
/api/contact	POST	Submit contact enquiry and send notification/confirmation emails	netlify/functions/contact.js
/api/careers	GET	Retrieve available job openings	netlify/functions/careers.js
/api/solutions	GET	Retrieve company solution data	netlify/functions/solutions.js
/api/careers-apply	POST	Submit job application and optional resume attachment	netlify/functions/careers-apply.js

Netlify API Routing

The netlify.toml file maps the /api/* path pattern to Netlify Functions using a rewrite:

```
[[redirects]]
  from = "/api/*"
  to = "/.netlify/functions/:splat"
  status = 200
```

Therefore, the deployed Netlify application is designed to expose the function endpoints through the /api path.

Local API Support

The included server.js explicitly handles /api/contact, /api/careers, and /api/careers-apply. The local server does not contain an explicit /api/solutions route, even though the solutions Netlify Function exists. This should be considered during local testing.

Health Check

The supplied project does not define an /api/health endpoint. A health endpoint should not be documented as available unless one is added to the application.

8. Security and Validation

Shared validation is implemented in lib/validation.js and reused by the contact and career application functions.

Protection	Implementation
HTML escaping	escapeHtml() protects generated email HTML from injected markup.
Input normalization	normalizeString() trims values and applies maximum lengths.
Email validation	isEmailValid() performs basic email format validation.
Honeypot	A hidden website field is rejected when populated, helping detect simple bots.

Rate limiting	20 requests per 60 seconds per client IP in the in-process limiter.
Request size	Local server limits request bodies to 5 MB.
Resume validation	PDF, DOC, DOCX MIME types; maximum 5 MB.
CORS	Allowed development origins are defined in validation.js.
Security headers	Local static responses include X-Frame-Options, X-Content-Type-Options and Referrer-Policy.
Credentials	SMTP credentials are read from environment variables rather than frontend code.

Security Recommendations for Production

- Keep .env out of source control and configure production secrets through the hosting provider.
- Use an email App Password or provider-specific secure SMTP authentication where supported.
- Consider centralized/distributed rate limiting for a high-traffic serverless deployment because the current limiter stores state in process memory.
- Review upload validation and consider stronger file-content checks if resume uploads become a significant attack surface.
- Review production CORS and security-header policy against the final domain and deployment architecture.

9. Project Structure

```
Afnamtech_Private_Limited-Website-main/
|
+-- .env.example
+-- .gitignore
+-- README.md
+-- package.json
+-- package-lock.json
+-- deno.lock
+-- server.js
+-- test-server.js
+-- netlify.toml
|
+-- lib/
|   +-- company.js
|   +-- validation.js
|
+-- netlify/
|   +-- functions/
|       +-- contact.js
|       +-- careers.js
|       +-- solutions.js
|       +-- careers-apply.js
|
+-- public/
|   +-- index.html
|   +-- our-story.html
|   +-- solutions.html
|   +-- what-we-offer.html
|   +-- career.html
|   +-- assets/
|   +-- css/
|   +-- js/
|   +-- react/
```

```
|
+-- scripts/
|   +-- build.mjs
|
+-- tests/
    +-- validation.test.js
```

Directory Summary

Directory/File	Responsibility
public/	Frontend pages, stylesheets, scripts, SVG assets, and React source
public/css/	Global and page-specific CSS
public/js/	Navigation, forms, page interactions, career loading, and animations
public/react/	React component source for contact/career widgets
netlify/functions/	Serverless API handlers
lib/	Shared company configuration and validation utilities
scripts/	Build automation for the React bundle
tests/	Automated validation/API-focused tests
server.js	Local static file server and local API adapter
netlify.toml	Netlify build, publish, function, and redirect configuration
.env.example	Template for SMTP environment variables

10. Detailed File Explanation

1. netlify/functions/contact.js

- Accepts POST requests for contact enquiries.
- Parses and validates JSON input.
- Rejects honeypot submissions and excessive request frequency.
- Checks SMTP configuration.
- Sends the enquiry to COMPANY_EMAIL.
- Sends a confirmation message to the visitor.

2. netlify/functions/careers.js

- Contains the current four job definitions.
- Returns the jobs as JSON with success and data properties.
- Does not use a database.

3. netlify/functions/solutions.js

- Contains six solution definitions.
- Returns solution information as JSON.
- Does not use a database.

4. netlify/functions/careers-apply.js

- Accepts job application submissions.
- Validates applicant fields.
- Validates an optional Base64 resume upload.
- Adds the resume as an email attachment.
- Sends the application to COMPANY_EMAIL.
- Sends an application-received confirmation to the candidate.

5. lib/company.js

- Stores company name, default company email, address, phone numbers, and social links.
- Provides centralized company configuration values.

6. lib/validation.js

- Defines maximum input lengths and allowed origins.
- Provides HTML escaping, normalization, email validation, client-IP extraction, CORS headers, JSON responses, rate limiting, and resume parsing.

11. Frontend, Build System, and Testing

Frontend JavaScript

File	Main Responsibility
public/js/form.js	Contact/career form submission, validation, resume Base64 conversion, popup behavior and related UI helpers
public/js/nav.js	Navigation, mobile menu/drawer, active navigation and interaction behavior
public/js/home.js	Home-page slider and timed slide changes
public/js/solutions.js	Solution card expand/collapse interaction
public/js/what-we-offer.js	Tool/card interactions and counters
public/js/our-story.js	Milestone/scroll-based visual behavior
public/js/career.js	Fetches /api/careers, filters jobs and controls application UI

CSS Organization

File	Purpose
global.css	Shared layout, typography, navigation, buttons, common components and responsive behavior
home.css	Home-page-specific styling
our-story.css	Our Story page styling
solutions.css	Solutions page styling
what-we-offer.css	What We Offer page styling
career.css	Career page styling

React and Build System

The project also contains React 19 components under public/react/. The scripts/build.mjs file uses esbuild to bundle public/react/index.jsx into public/js/react-widgets.js. The main HTML pages currently rely on their vanilla JavaScript files for the visible site flows, so the React source represents an additional implementation layer that should be kept only if it is intentionally used.

Testing

The supplied tests/validation.test.js file contains seven focused tests covering HTML escaping, normalization, email validation, JSON response headers, PDF upload parsing, unsupported upload rejection, and rate limiting. These tests are designed to run with the Node.js built-in test runner.

12. Deployment Process

The project is configured for Netlify deployment.

Step 1: Push Project to GitHub

```
git init
git add .
git commit -m "Initial Commit - Afnamtech Website"
git branch -M main
git remote add origin <repository-url>
git push -u origin main
```

Step 2: Configure Netlify

- Import the GitHub repository into Netlify.
- Build command: npm run netlify-build.
- Publish directory: public.
- Functions directory: netlify/functions.

Step 3: Add Environment Variables

Variable	Value
SMTP_HOST	SMTP provider host
SMTP_PORT	SMTP provider port
SMTP_USER	SMTP sending account
SMTP_PASS	SMTP password/App Password
COMPANY_EMAIL	Company notification mailbox

Step 4: Deploy and Verify

- Open the deployed website.
- Verify all five pages and navigation routes.
- Open the Career page and confirm jobs load.
- Submit a contact enquiry and verify the company notification and visitor confirmation.
- Submit a career application and verify the company application email and candidate confirmation.
- Test a valid resume and invalid/oversized upload cases.

Live Application Information

No production website URL is defined in the supplied project files. This documentation therefore does not invent or claim a live URL.

13. Implementation Notes and Conclusion

Important Implementation Notes

- The project does not contain a database layer. Contact enquiries and career applications are delivered through email rather than stored in a database.
- The current job listings and solution catalog are static JavaScript data.
- The Netlify configuration provides the intended serverless deployment path.
- The local Node server reuses the Netlify function handlers for contact, careers, and career applications.
- The solutions Netlify Function exists, but the local server does not explicitly map /api/solutions.
- The project contains both vanilla JavaScript and React implementations. The visible HTML pages primarily use the vanilla implementation.
- The repository includes test-server.js, but the main local server is server.js.

Recommended Final Verification

- Confirm the final company email address and SMTP account.
- Confirm the final company address, phone numbers, and social-media links.

- Remove any unused or duplicate frontend implementation if React is not required.
- Add a dedicated health endpoint only if operational monitoring is needed.
- Consider centralized rate limiting for production-scale traffic.
- Perform a complete mobile, accessibility, link, form, and deployment test before launch.

Conclusion

The Afnamtech Private Limited Website is a corporate banking-technology website with a static frontend, a lightweight Node.js local server, and Netlify serverless functions for backend operations. Its primary dynamic capabilities are contact enquiries, career listing retrieval, and job application processing with email notifications and confirmations. The project includes reusable validation utilities, basic anti-spam protections, resume validation, SMTP-based email delivery, Netlify deployment configuration, and automated tests. The supplied source is suitable as a foundation for deployment after the final configuration, content, route, and production-security checks are completed.